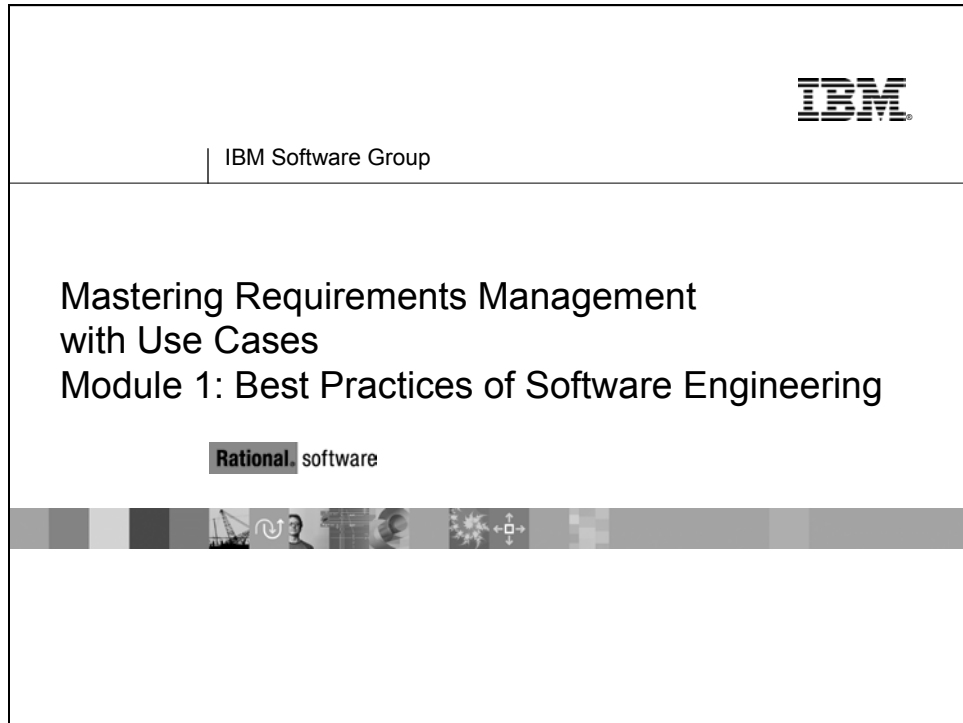


► ► ► **Module 1**
Best Practices of Software Engineering



Topics

Objectives.....	1-2
Symptoms of Software Development Problems.....	1-3
Practice 1: Develop Iteratively	1-5
Practice 2: Manage Requirements	1-10
Practice 3: Use Component Architectures.....	1-12
Practice 4: Model Visually (UML).....	1-15
Practice 5: Continuously Verify Quality.....	1-17
Practice 6: Manage Change	1-21
Change Request Management Concepts	1-22
A Team-Based Definition of Process	1-26
Iterations and Phases	1-29

Objectives

Objectives

- ♦ List symptoms of software development problems.
- ♦ Define the six Best Practices.
- ♦ Describe activities for solving software engineering problems for each Best Practice.
- ♦ Describe the Rational Unified Process® (RUP®) within the context of the six Best Practices.

Symptoms of Software Development Problems

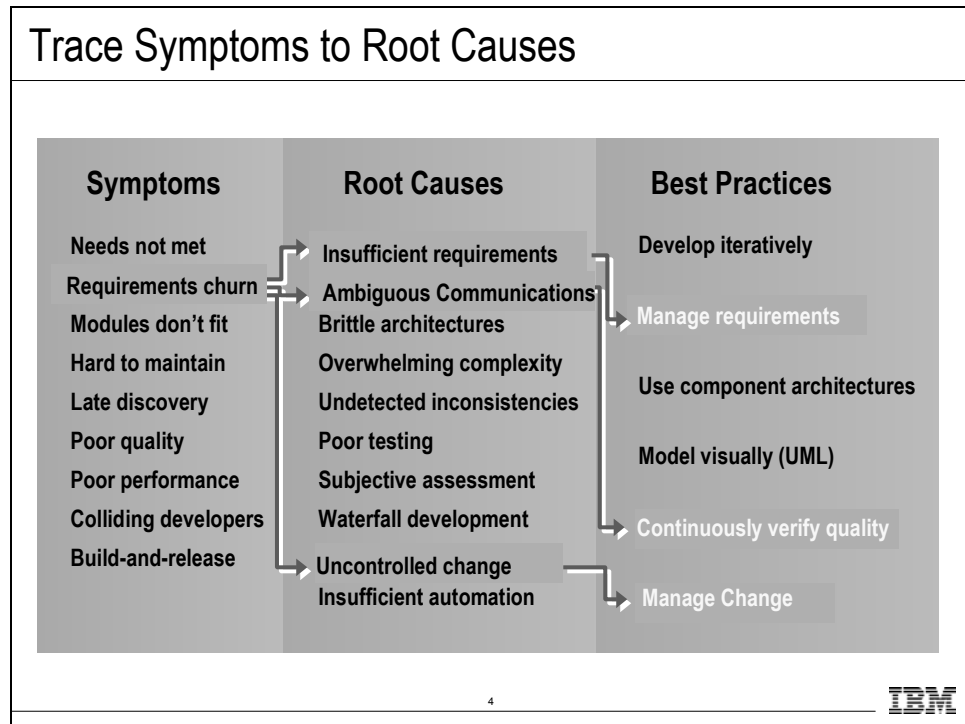
Symptoms of Software Development Problems

- ✓ User or business needs not met.
- ✓ Requirements churn.
- ✓ Modules don't integrate.
- ✓ Hard to maintain.
- ✓ Late discovery of flaws.
- ✓ Poor quality or end-user experience.
- ✓ Poor performance under load.
- ✓ No coordinated team effort.
- ✓ Build-and-release issues.

3



Trace Symptoms to Root Causes



Treat the root causes, and you'll eliminate the symptoms. Eliminate the symptoms, and you'll be in a much better position to develop quality software in a repeatable and predictable fashion.

Best practices are a set of commercially proven approaches to software development, which, when used in combination, strike at the root causes of software development problems. These are so-called "best practices," not so much because we can precisely quantify their value, but rather because they are observed to be commonly used in the industry by successful organizations.

The best practices are harvested from thousands of customers on thousands of projects and from industry experts.

Practice 1: Develop Iteratively

Practice 1: Develop Iteratively



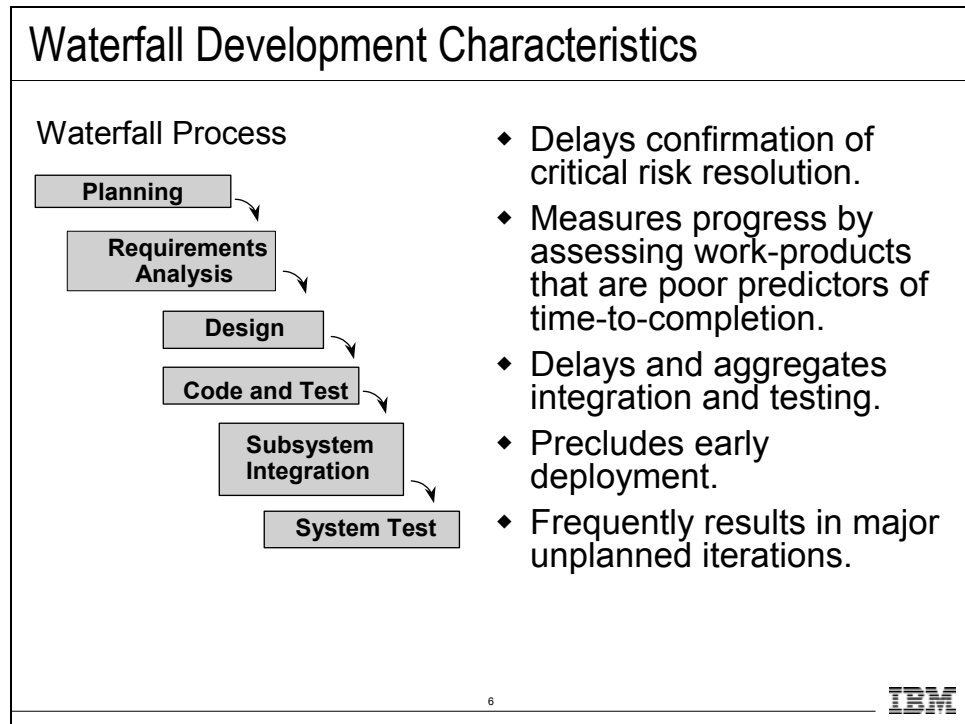
5



Developing iteratively is a technique that is used to deliver the functionality of a system in a successive series of releases of increasing completeness. Each release is developed in a specific, fixed time period called an “iteration.”

Each iteration is focused on defining, analyzing, designing, building and testing some set of requirements.

Waterfall Development Characteristics

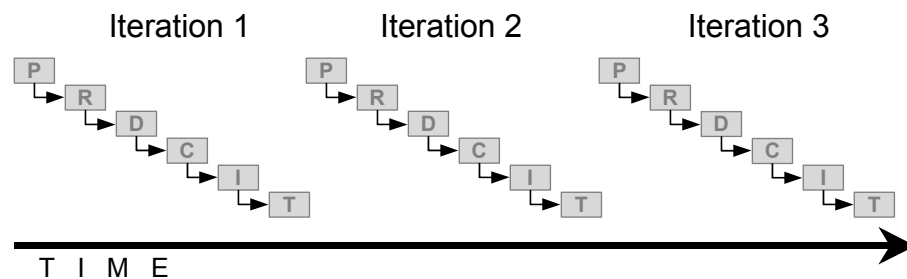


Waterfall is conceptually straightforward because it produces a single deliverable for each step (requirements, analysis model, design model, code, etc), resulting in a single release. The fundamental problem is that it pushes risk forward in time, where it's costly to undo mistakes from earlier phases. An initial design will likely be flawed with respect to its key requirements, and furthermore, the late discovery of design defects tends to result in costly overruns or project cancellation. The waterfall approach tends to mask the real risks to a project until it is too late to do anything meaningful about them.

Iterative Development Characteristics

Iterative Development Characteristics

- ♦ Resolves major risks before making large investments.
- ♦ Enables early user feedback.
- ♦ Makes testing and integration continuous.
- ♦ Focuses project short-term objective milestones.
- ♦ Makes possible deployment of partial implementations.



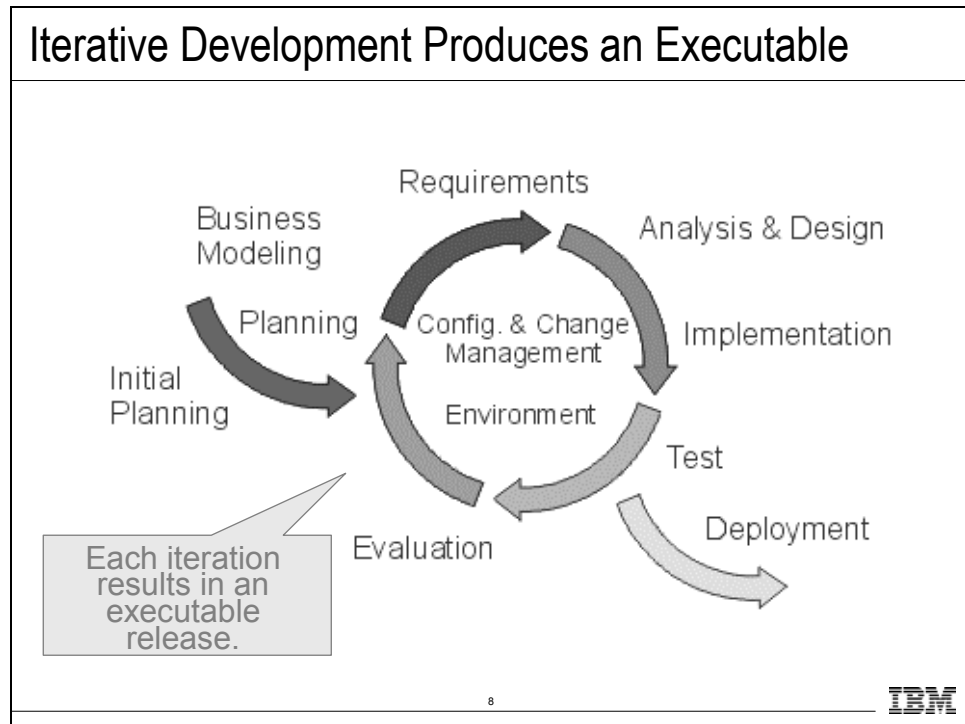
7

IBM

Iterative processes were developed in response to these waterfall characteristics. With an iterative process, the waterfall steps are applied *iteratively*. Instead of developing the whole system in lock step, an increment (that is, a subset of system functionality) is selected and developed, then another increment, and so on. The selection of the first increment to be developed is based on risk, the highest priority risks first. To address the selected risk(s), choose a subset of use cases. Develop the *minimal* set of use cases that will allow objective verification, for example, through a set of executable tests, of the risks that you have chosen. Then select the next increment to address the next highest risk, and so on. Thus you apply the waterfall within each iteration, and the system evolves incrementally.

P: Planning
 R: Requirements analysis
 D: Design
 C: Code and unit test
 I: Integration
 T: Test

Iterative Development Produces an Executable

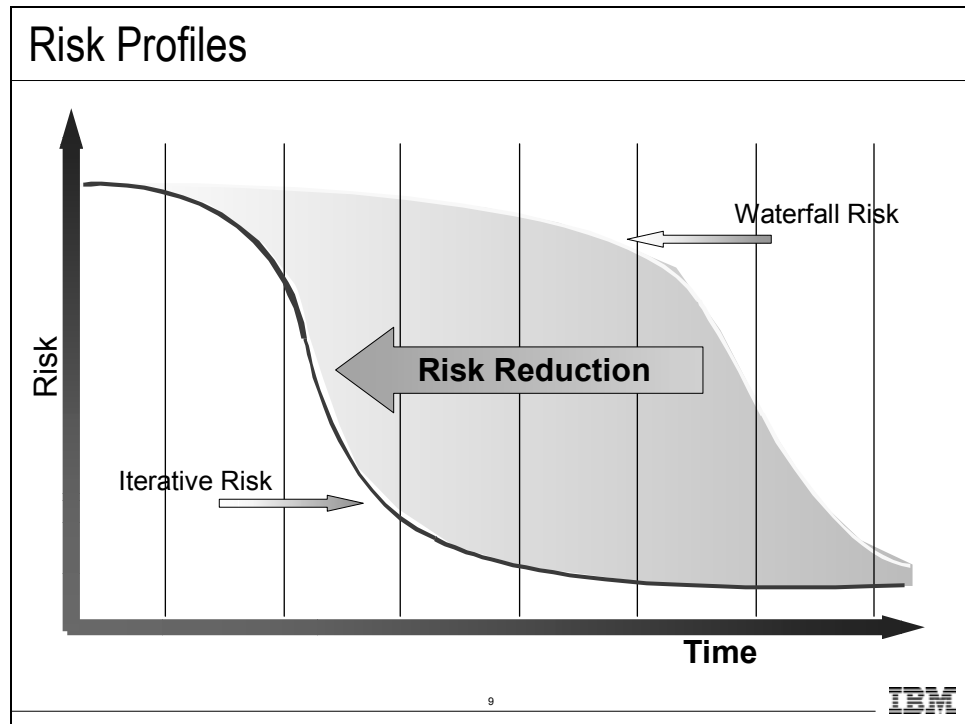


The earliest iterations address the greatest risks. Each iteration produces an executable release. Each iteration includes integration and test.

Iterations help:

- Resolve major risks before making large investments.
- Enable early user feedback.
- Make testing and integration continuous.
- Focus project short-term objective milestones.
- Make possible deployment of partial implementations.

Risk Profiles



Iterative development produces the architecture first, allowing integration to occur “as the verification activity” of the design phase and allowing design flaws to be detected and resolved earlier in the lifecycle. Continuous integration throughout the project replaces the “big bang” integration at the end of a project.

Iterative development also provides much better insight into quality, because of system characteristics that are largely inherent in the architecture. For example, performance, fault tolerance, and maintainability are tangible earlier in the process. Thus, issues are still correctable without jeopardizing target costs and schedules.

Practice 2: Manage Requirements

Practice 2: Manage Requirements

Best Practices
Process Made Practical
Develop Iteratively
Manage Requirements
Use Component Architectures
Model Visually (UML)
Continuously Verify Quality
Manage Change

10

IBM

A report from the Standish Group confirms that a distinct minority of software development projects is completed on-time and on-budget. In their report, the success rate was only 28 percent, while challenged projects (operational, but late and over-budget) accounted for 49 percent.

Cancelled projects accounted for 23 percent. These failures are attributed to lack of executive management support, lack of user involvement, inexperienced project managers, ill-defined business objectives, poor scope management, no standard software infrastructure, no firm basic requirements, no formal methodology, and unreliable estimates. (Source: Chaos 2001 Report, <http://www.standishgroup.com>).

Requirements Management

Requirements Management

- ♦ Making sure you solve the right problem and build the right system.
- ♦ By taking a systematic approach to:
 - Understanding the problem.
 - Eliciting, organizing, and documenting the requirements.
 - Managing the changing requirements of a software application.



Practice 3: Use Component Architectures

Practice 3: Use Component Architectures

Best Practices
Process Made Practical

Develop Iteratively
Manage Requirements
Use Component Architectures
Model Visually (UML)
Continuously Verify Quality
Manage Change

12

IBM

Software architecture is the development product that gives the highest return on investment with respect to quality, schedule, and cost, according to the authors of *Software Architecture in Practice* (Len Bass, Paul Clements & Rick Kazman [1998] Addison-Wesley).

The Software Engineering Institute (SEI) has an effort underway called the Architecture Tradeoff Analysis (ATA) Initiative to focus on software architecture, a discipline much misunderstood in the software industry. The SEI has been evaluating software architectures for some time and would like to see architecture evaluation in wider use. By performing architecture evaluations, AT&T reported a 10 percent productivity increase (from *news@sei*, Vol. 1, No. 2).

Resilient Component-Based Architectures

Resilient Component-Based Architectures

- ◆ Resilient
 - Meets current and future requirements.
 - Improves extensibility.
 - Enables reuse.
 - Encapsulates system dependencies.
- ◆ Component-based
 - Reuse or customize components.
 - Select from commercially available components.
 - Evolve existing software incrementally.

13



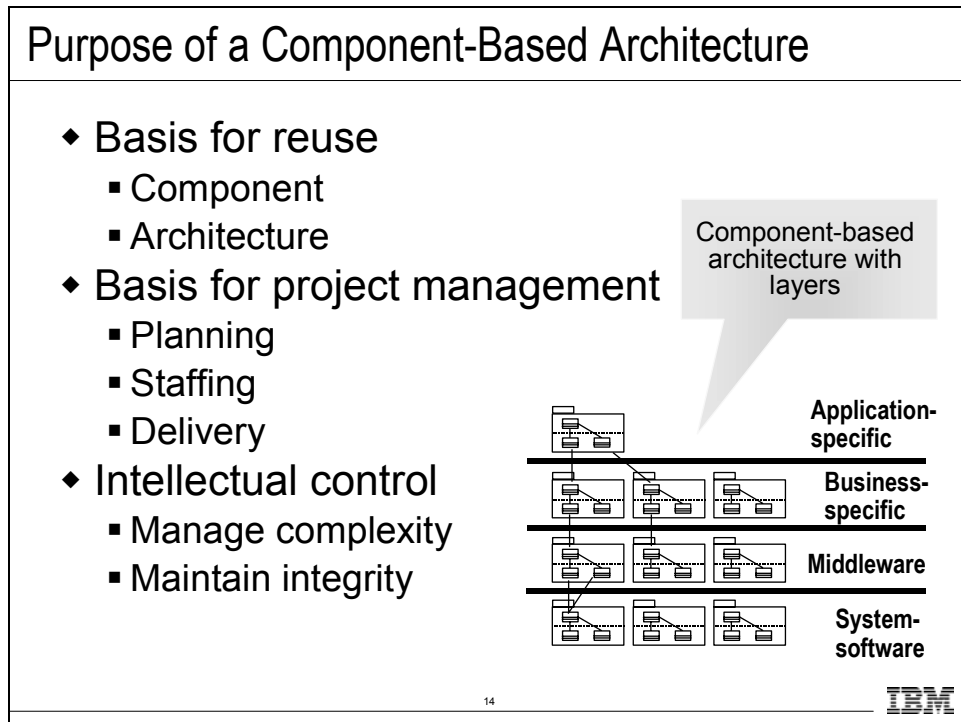
Architecture is a part of design. It is about making decisions on how the system is to be built. But it is not all of the design. It stops at the major abstractions or, in other words, the elements that have some pervasive and long-lasting effect on the system's performance and ability to evolve.

A software system's architecture is perhaps the most important aspect that can be used to control the iterative and incremental development of a system throughout its lifecycle.

The most important property of an architecture is resilience—flexibility in the face of change. To achieve it, architects must anticipate evolution in both the problem domain and implementation technologies to produce a design that can gracefully accommodate such changes.

Key techniques are abstraction, encapsulation, and object-oriented analysis and design. The result is that applications are fundamentally more maintainable and extensible.

Purpose of a Component-Based Architecture



Definition of a (Software) Component:

RUP Definition: A nontrivial, nearly independent, and replaceable part of a system that fulfills a clear function in the context of a well-defined architecture. A component conforms to and provides the physical realization of a set of interfaces.

UML Definition: A physical, replaceable part of a system that packages implementation, and conforms to and provides the realization of a set of interfaces. A component represents a physical piece of implementation of a system, including software code (source, binary, or executable) or equivalents, such as scripts or command files.

Practice 4: Model Visually (UML)

Practice 4: Model Visually (UML)



15

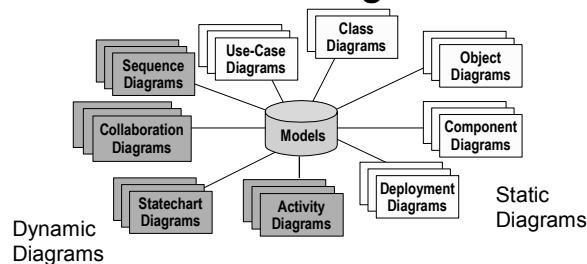


A **model** is a simplification of reality that provides a complete description of a system from a particular perspective. We build models so that everyone can understand the system to be built. Models are built for complex systems because we cannot comprehend any such system in its entirety.

Why Model Visually?

Why Model Visually?

- ◆ Capture structure and behavior.
- ◆ Show how system elements fit together.
- ◆ Keep design and implementation consistent.
- ◆ Hide or expose details as appropriate.
- ◆ Promote unambiguous communication.



UML provides one language for all practitioners.

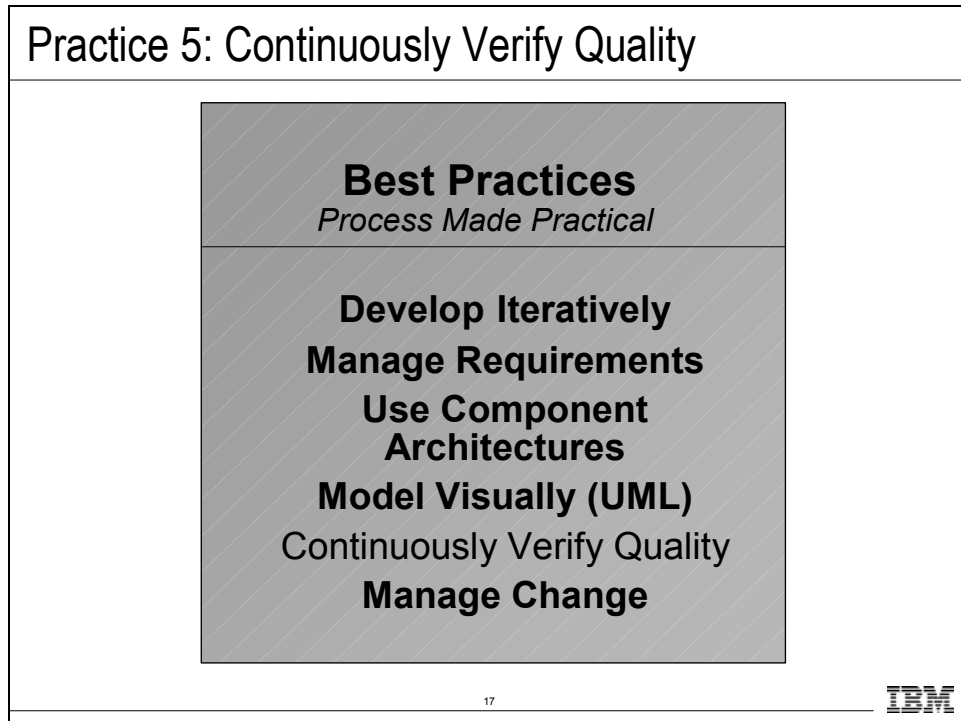
16

IBM

Modeling is important because it helps the development team visualize, specify, construct, and document the structure and behavior of a system's architecture. Using a standard modeling language, such as the Unified Modeling Language (UML), different members of the development team can communicate their decisions unambiguously to one another.

Using visual modeling tools facilitates the management of these models, letting you hide or expose details as necessary. Visual modeling also helps you maintain consistency among a system's artifacts—its requirements, designs, and implementations. In short, visual modeling helps improve a team's ability to manage software complexity.

Practice 5: Continuously Verify Quality



Quality, as used within RUP, is defined as "The characteristic of having demonstrated the achievement of producing a product which meets or exceeds agreed-upon requirements, as measured by agreed-upon measures and criteria, and is produced by an agreed-upon process." Given this definition, achieving quality is not simply "meeting requirements" or producing a product that meets user needs and expectations. Quality also includes identifying the measures and criteria (to demonstrate the achievement of quality) and the implementation of a process to ensure that the resulting product has achieved the desired degree of quality (and can be repeated and managed).

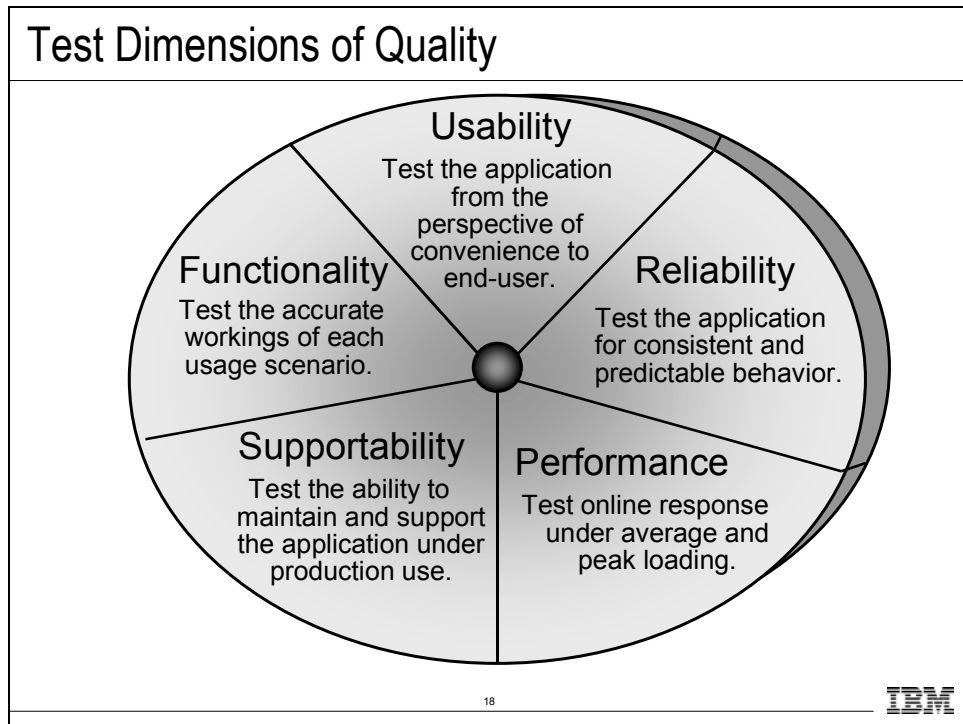
In many organizations, software testing accounts for 30 percent to 50 percent of software development costs. Yet most people believe that software is not well-tested before it is delivered.

This contradiction is rooted in two clear facts.

First, testing software is enormously difficult. The different ways a particular program can behave are almost infinite.

Second, testing is typically done without a clear methodology and without the required automation or tool support. While the complexity of software makes complete testing an impossible goal, a well-conceived methodology and use of state-of-the-art tools can greatly improve the productivity and effectiveness of the software testing.

Test Dimensions of Quality



Functional testing verifies that a system executes the required use-case scenarios as intended. Functional tests may include the testing of features, usage scenarios, and security.

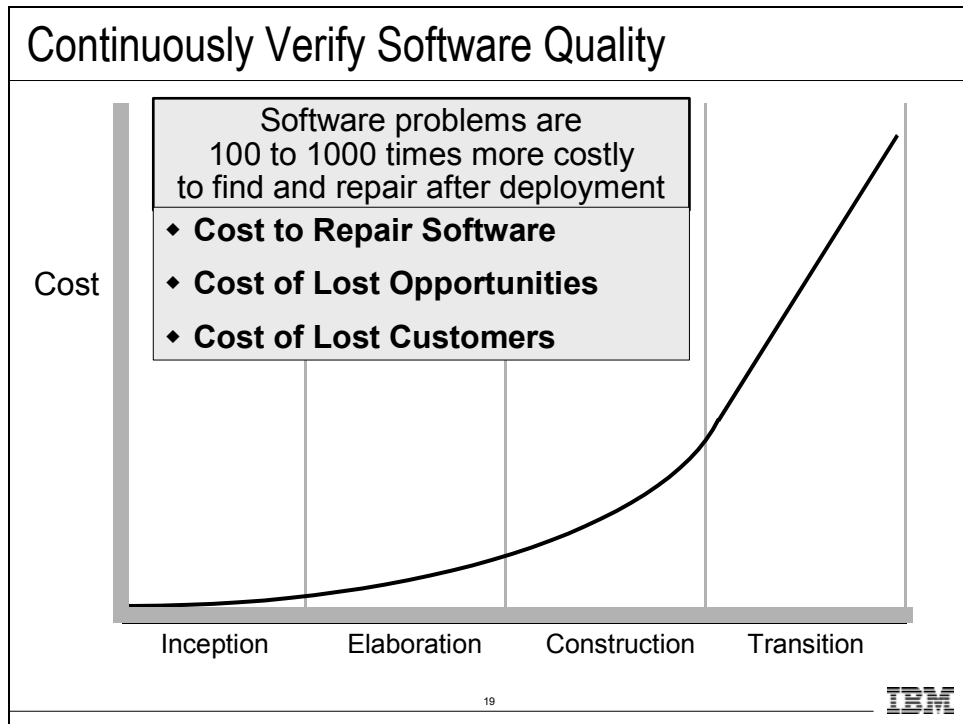
Usability testing evaluates the application from the user's perspective. Usability tests focus on human factors, aesthetics, consistency in the user interface, online and context-sensitive help, wizards and agents, user documentation, and training materials.

Reliability testing verifies that the application performs reliably and is not prone to failures during execution (crashes, hangs, memory leaks). Effective reliability testing requires specialized tools. Reliability tests include integrity, structure, stress, contention, and volume tests.

Performance testing checks that the target system works functionally and reliably under production load. Performance tests include benchmark tests, load tests, and performance profile tests.

Supportability testing verifies that the application can be deployed as intended. Supportability tests include installation and configuration tests.

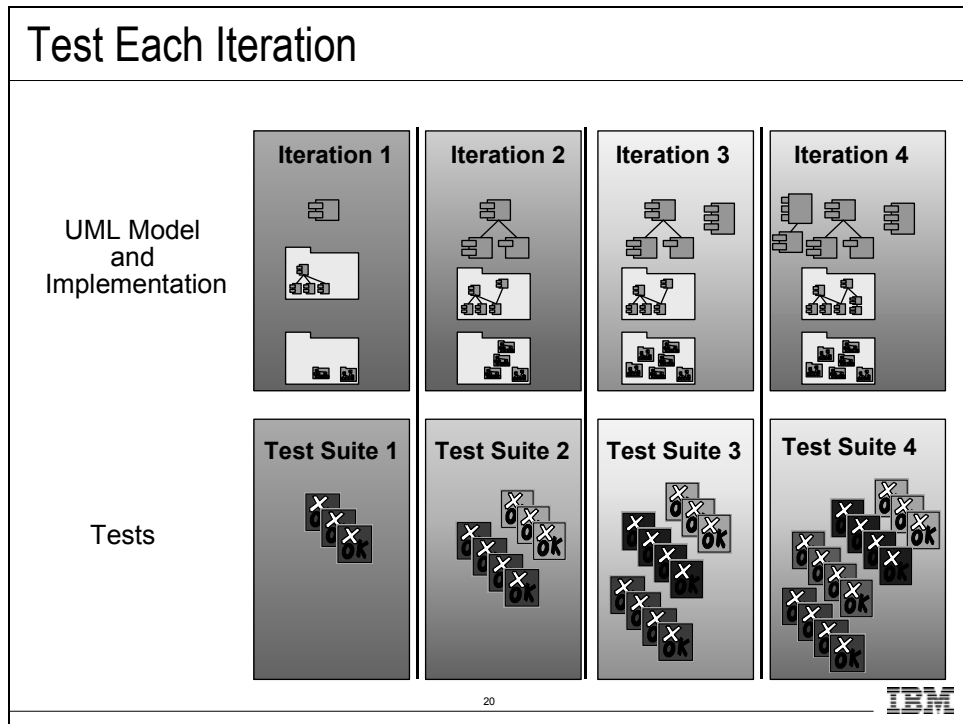
Continuously Verify Software Quality



This principle is driven by a fundamental and well-known property of software development: it's a lot less expensive to correct defects during development than to correct them after deployment.

- Tests for key scenarios ensure that all requirements are properly implemented.
- Poor application performance hurts as much as poor reliability.
- Verify software reliability: memory leaks, bottlenecks.
- Test every iteration and automate test.

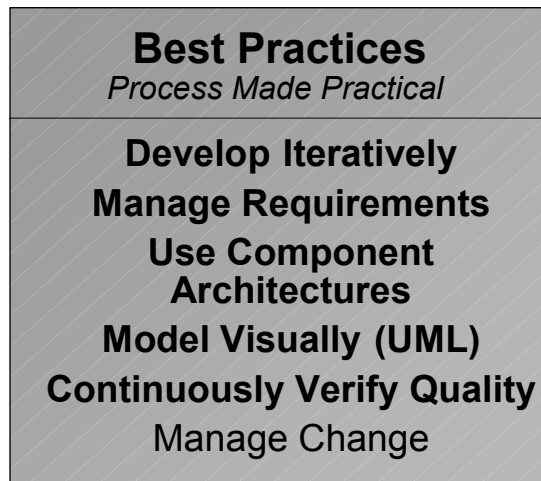
Test Each Iteration



In each iteration, automated tests are created that test the requirements addressed in that iteration. As new requirements are added in subsequent iterations, new tests are generated and run. At times, a requirement may be changed in a later iteration. In that case, the tests associated with the changed requirement may be modified or simply regenerated by an automated tool.

Practice 6: Manage Change

Practice 6: Manage Change



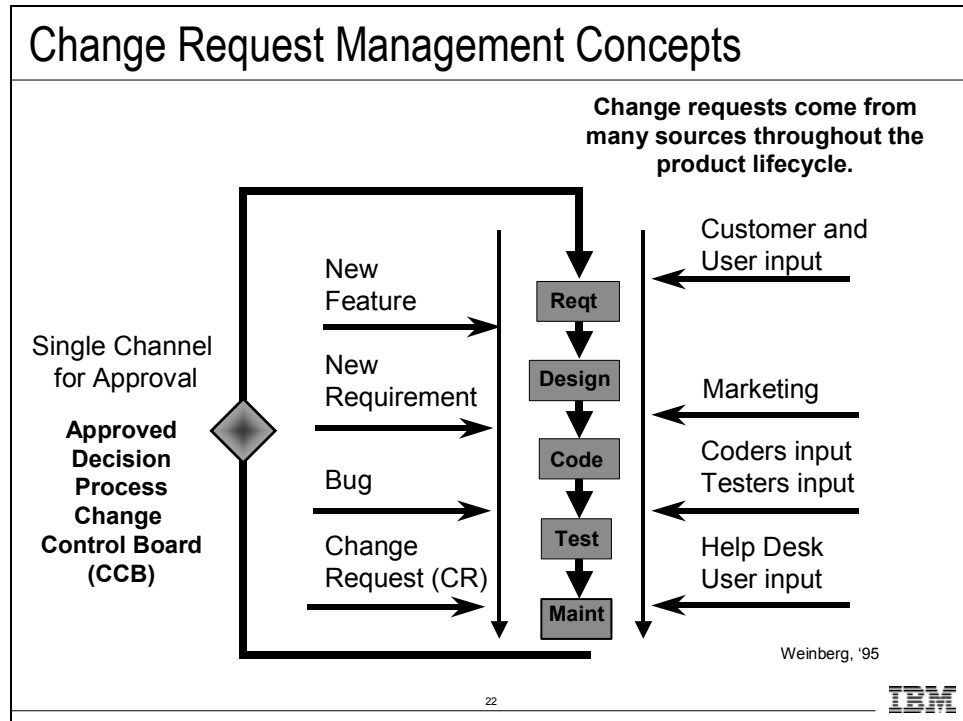
21



As indicated earlier, you cannot stop change from being introduced into a project. However, you must control how and when changes are introduced into project artifacts and who introduces the changes. As well, you must synchronize change across development teams and locations.

Unified Change Management (UCM) is Rational Software's approach to managing change in software system development, from requirements to release.

Change Request Management Concepts



Change Request Management (CRM) addresses the organizational infrastructure required to assess the cost and schedule impact of a requested change to the existing product. CRM addresses the workings of a Change Review Team or Change Control Board.

Configuration Status Accounting (Measurement) is used to describe the “state” of the product based on the type, number, rate, and severity of defects found and fixed during the course of product development. Metrics derived under this aspect, either through audits or raw data, are useful in determining the overall completeness of the project.

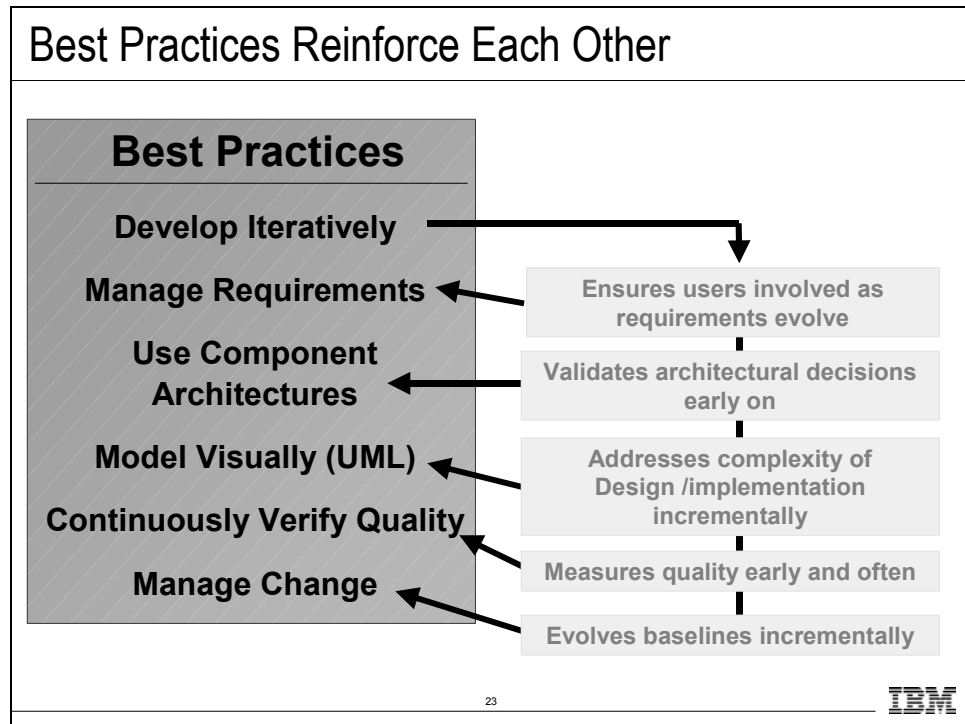
Configuration Management (CM) describes the product structure and identifies its constituent configuration items that are treated as single versionable entities in the configuration management process. CM deals with defining configurations, building and labeling, and collecting versioned artifacts into constituent sets and maintaining traceability between these versions.

Change Tracking describes what is done to components for what reason and at what time. It serves as the history and rationale of changes. It is quite separate from assessing the impact of proposed changes as described under Change Request Management.

Version Selection is to ensure that the right versions of configuration items are selected for change or implementation. Version selection relies on a solid foundation of “configuration identification.”

Software Manufacture covers the need to automate the steps to compile, test, and package software for distribution.

Best Practices Reinforce Each Other

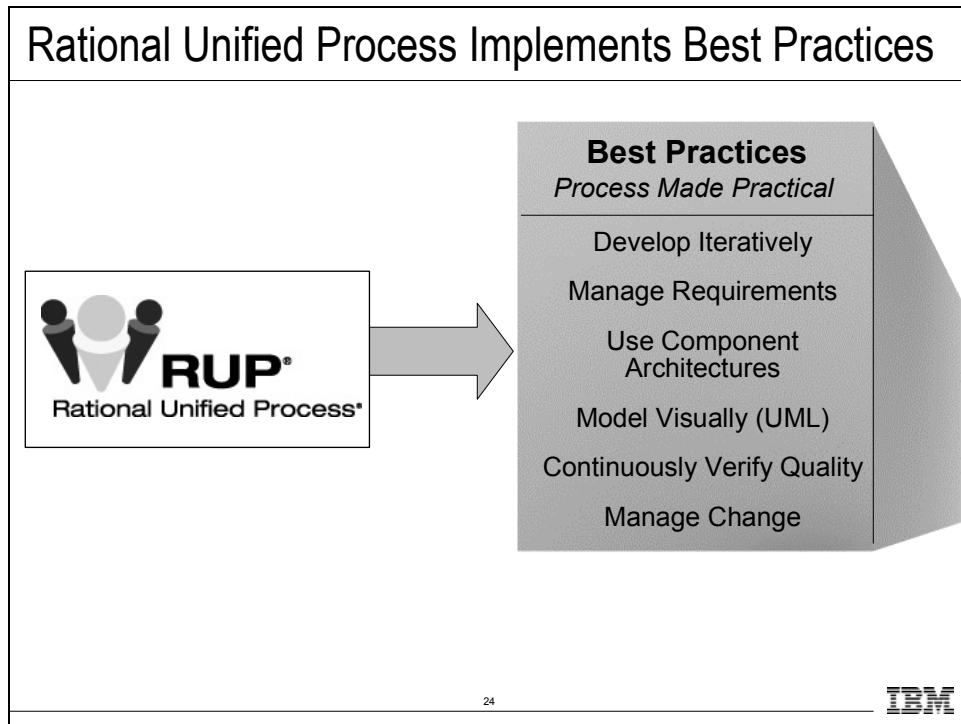


In the case of our six Best Practices, the whole is greater than the sum of the parts. Each Best Practice reinforces and, in some cases, enables the others. The slide shows just one example:

Iterative development leverages the other five Best Practices.

However, each of the other five practices also enhances iterative development. For example, iterative development done without adequate requirements management can easily fail to converge on a solution. Requirements change at will, users can't agree, and the iterations can go on forever. When requirements are managed, this is less likely to happen. Changes to requirements are visible, and the impact on the development process is assessed before they are accepted. Convergence on a stable set of requirements is assured. Similarly, each pair of Best Practices provides mutual support. Although it is possible to use one Best Practice without the others, it is not recommended, since the resulting benefits are significantly decreased.

Rational Unified Process Implements Best Practices



Why have a process? A process:

- Provides guidelines for efficient development of quality software.
- Reduces risk and increases predictability.
- Promotes a common vision and culture.
- Captures and institutionalizes best practices.

Rational Unified Process is a generic business process for object-oriented software engineering. It describes a family of related software engineering processes sharing a common structure and a common process architecture. It provides a disciplined approach to assigning tasks and responsibilities within a development organization. Its goal is to ensure the production of high-quality software that meets the needs of its end users within a predictable schedule and budget. Rational Unified Process captures the best practices in modern software development in a form that can be adapted for a wide range of projects and organizations.

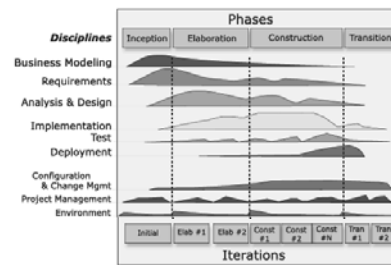
The UML provides a standard for the artifacts of development (semantic models, syntactic notation, and diagrams): the things that must be controlled and exchanged. The UML is not a standard for the development *process*.

Despite all of the value that a common modeling language brings, you cannot achieve successful development of today's complex systems with just the UML. Successful development also requires employing an equally robust development process.

Achieve Best Practices

Achieve Best Practices

- ◆ Iterative approach
- ◆ Guidance for activities and artifacts
- ◆ Process focus on architecture
- ◆ Use cases which drive design and implementation
- ◆ Models which abstract the system



25



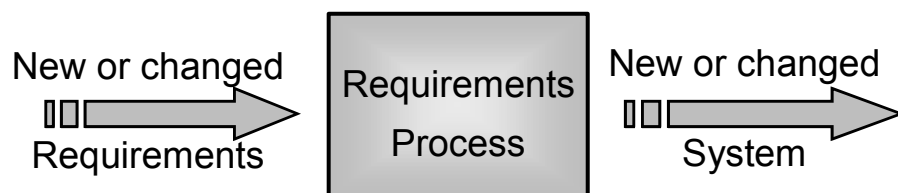
Examples:

- The dynamic structure (phases and iterations) of Rational Unified Process creates the basis of iterative development.
- The Project Management discipline describes how to set up and execute a project using phases and iterations.
- The Use-Case Model of the Requirements discipline and the Risk List determine what functionality you implement in an iteration.
- The Workflow Details of the Requirements discipline show the activities and artifacts that make requirements management possible.
- The iterative approach allows you to progressively identify components, decide which one to develop, which one to reuse, and which one to buy.
- The Unified Modeling Language (UML) used in the process represents the basis of Visual Modeling and has become the de facto modeling language standard.
- The focus on software architecture allows you to articulate the structure: the components, the ways in which they integrate, and the fundamental mechanisms and patterns by which they interact.

A Team-Based Definition of Process

A Team-Based Definition of Process

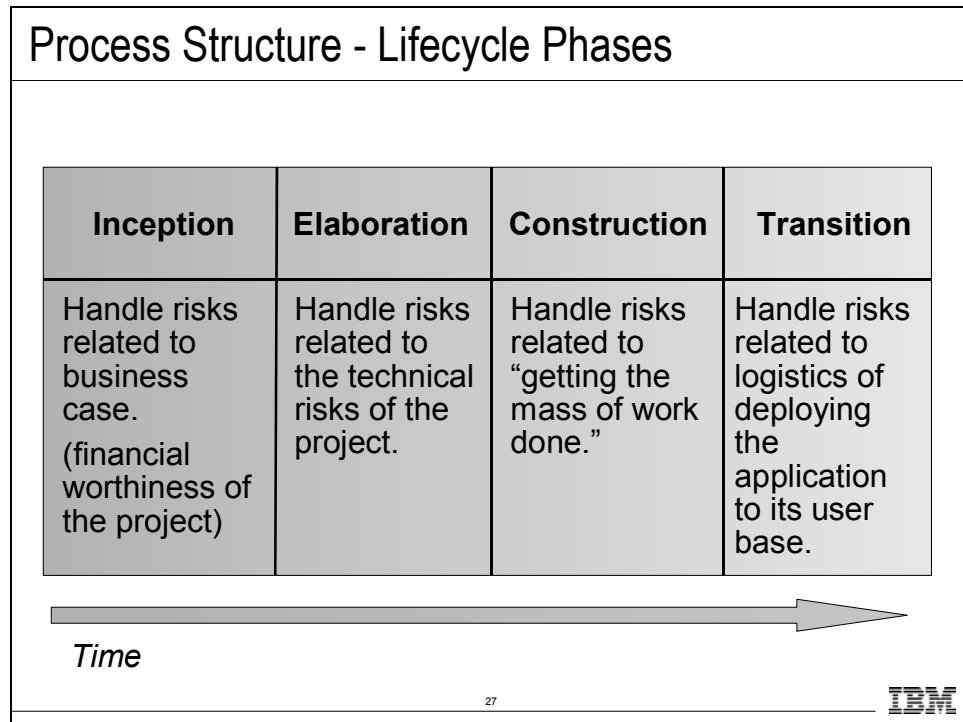
A process defines **Who** is doing **What**,
When, and **How** in order to reach a certain
goal.



26

IBM

Process Structure - Lifecycle Phases



During **Inception**, define the scope of the project, what is included, and what is not. Do this by identifying all the actors and use cases and by drafting the most essential use cases. A business plan is developed to determine whether resources should be committed to the project.

At the end of Inception you will have identified 100% of the use cases that you could identify at this stage. But this does not mean that some use cases will not show up later as you are getting deeper into the design.

During **Elaboration**, focus on two things: getting a good grasp of the requirements and establishing an architectural baseline. If you have a good grasp of the requirements and the architecture, you can eliminate the technical risks and have a good idea what amount of work remains. You can make detailed cost/resource estimations at the end of Elaboration.

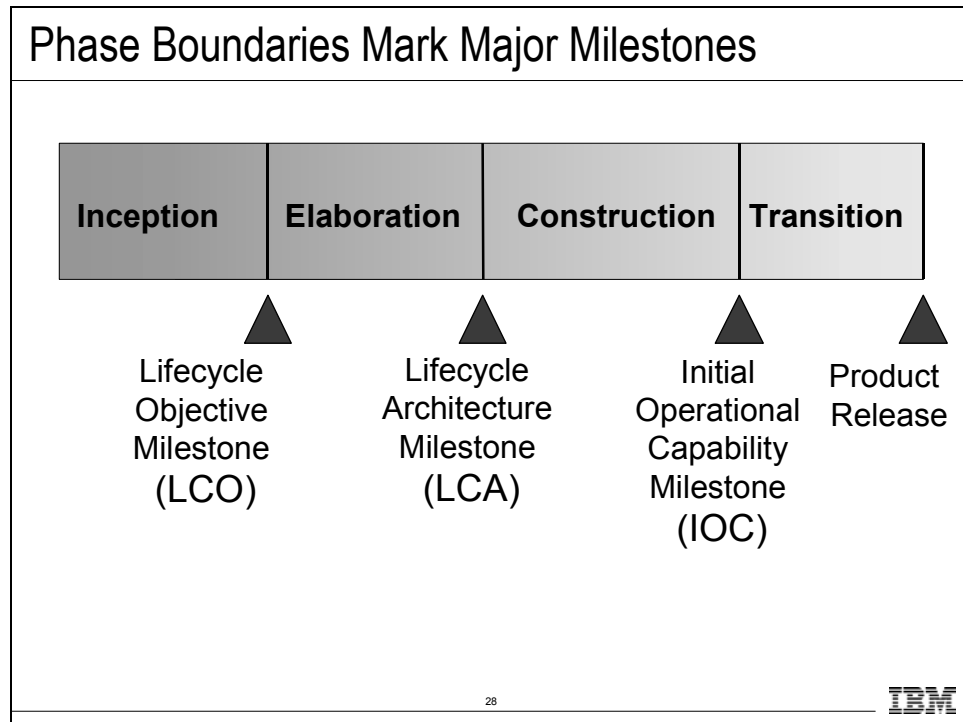
At the end of Elaboration you have a working version of the system that implements key use-case scenarios and stresses the most critical parts of the architecture (thereby removing the technical risks to the project).

During **Construction**, build the product in several iterations up to a beta release. Each iteration implements additional use-case scenarios.

During **Transition**, transition the product to the end user and focus on end user training, installation, and support.

The amount of time spent in each phase varies. For a complex project with a lot of technical unknowns and unclear requirements, Elaboration may include three to five iterations. For a simple project, where requirements are known and the architecture is simple, Elaboration may include only a single iteration.

Phase Boundaries Mark Major Milestones



At each of the major milestones, we review the project and decide whether to proceed with the project as planned, to abort the project, or to revise it. The criteria used to make this decision vary by phase.

LCO: *scope agreed upon and risks understood and reasonable.*

LCA: *high risks addressed and architecture stable.*

IOC: *product is complete and quality acceptable.*

The evaluation criteria for the inception phase (LCO) include:

Stakeholder concurrence on scope definition and cost/schedule estimates; requirements understanding as evidenced by the fidelity of the primary use cases; credibility of cost/schedule estimates, priorities, risks, and development process; depth and breadth of any architectural prototype; actual expenditures versus planned expenditures.

The evaluation criteria for the elaboration phase (LCA) include:

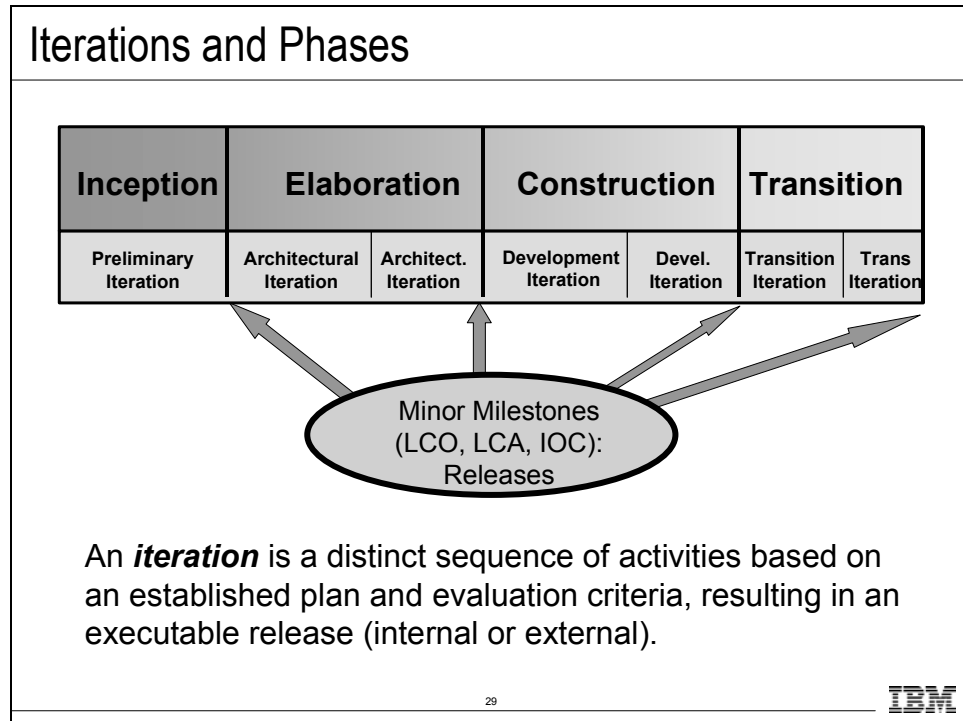
Stability of the product vision and architecture; resolution of major risk elements; adequate planning and reasonable estimates for project completion; stakeholder acceptance of the product vision and project plan; acceptable expenditure level.

The evaluation criteria for the construction phase (IOC) include:

Stability and maturity of the product release (that is, is it ready to be deployed?); readiness of the stakeholders for the transition; acceptable expenditure level.

At the end of the transition phase, we decide whether to release the product. We base this primarily on the level of user satisfaction achieved during the transition phase. Often this milestone coincides with the initiation of another development cycle to improve or enhance the product. In many cases, this new development cycle may already be underway.

Iterations and Phases



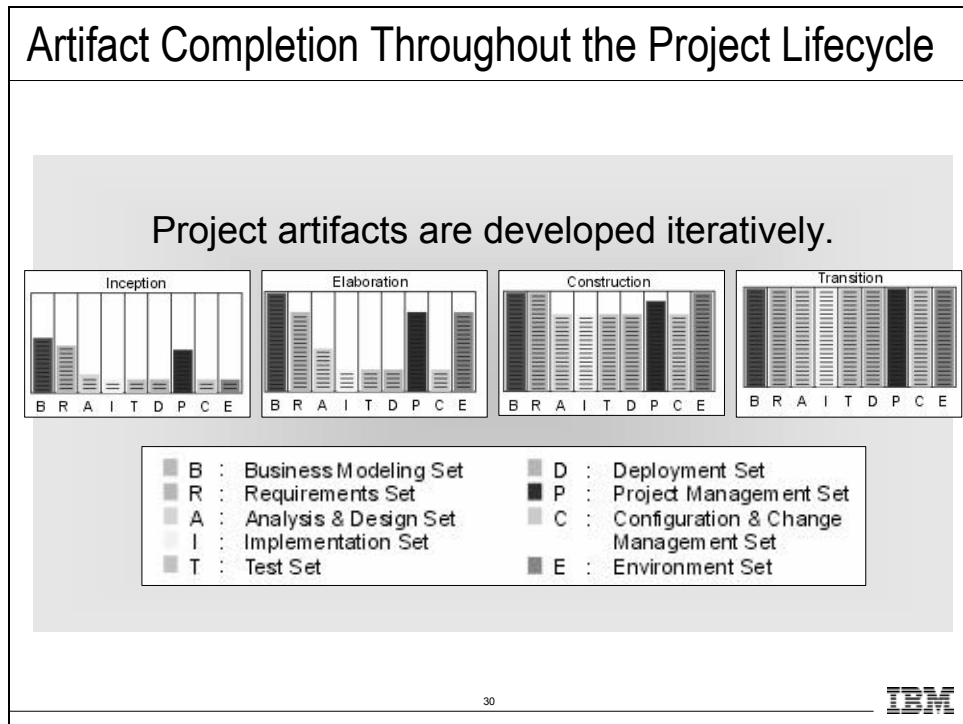
There is a series of iterations within each phase. The number of iterations per phase varies. Each iteration results in an executable release encompassing larger and larger subsets of the final application.

An internal release is kept within the development environment and (optionally) demonstrated to the stakeholder community. Provide stakeholders (usually users) with an external release for installation in their environment. External releases are much more expensive (they require user documentation and technical support) and normally occur only during the transition phase.

The end of an iteration marks a minor milestone. At this point, assess technical results and revise future plans as necessary.

The end of a phase marks a different milestone (but still generates an executable release). Specifically, the LCO, LCA, IOC and final release milestones.

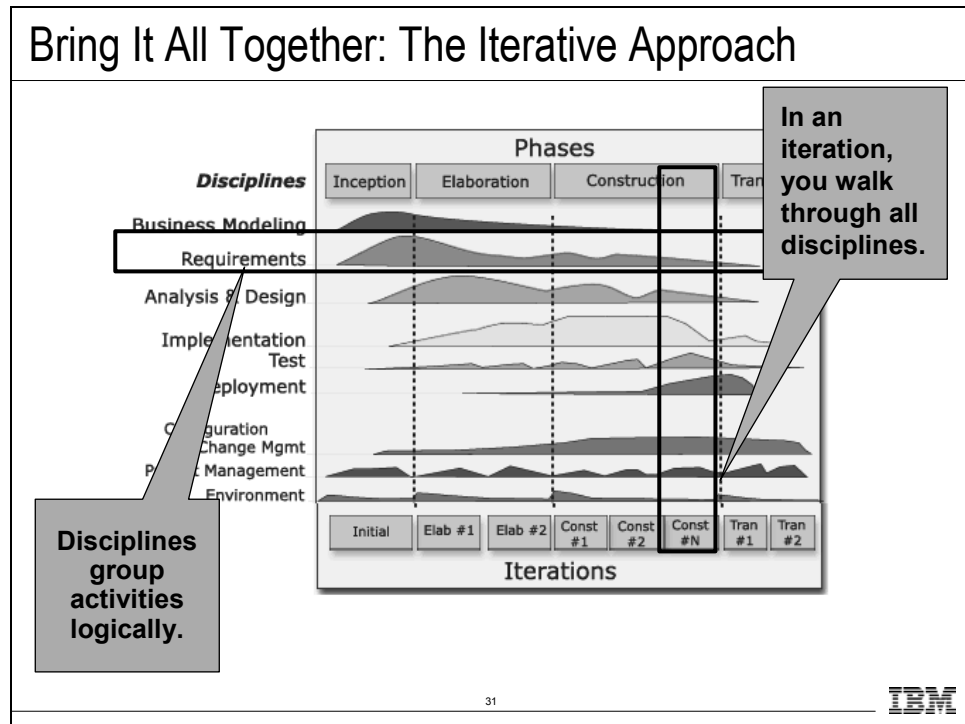
Artifact Completion Throughout the Project Lifecycle



In an iterative process, your artifact set is never developed all at once. Leaving an artifact in an incomplete state seems counter intuitive. This is why it is conceptually one of the most difficult things for most people to accept when moving to iterative development.

The key is to identify specific goals to be achieved for each iteration. A goal may be to prove the architecture or to deliver specific features. You then detail only the software requirements that are necessary in order for the goal to be achieved.

Bring It All Together: The Iterative Approach

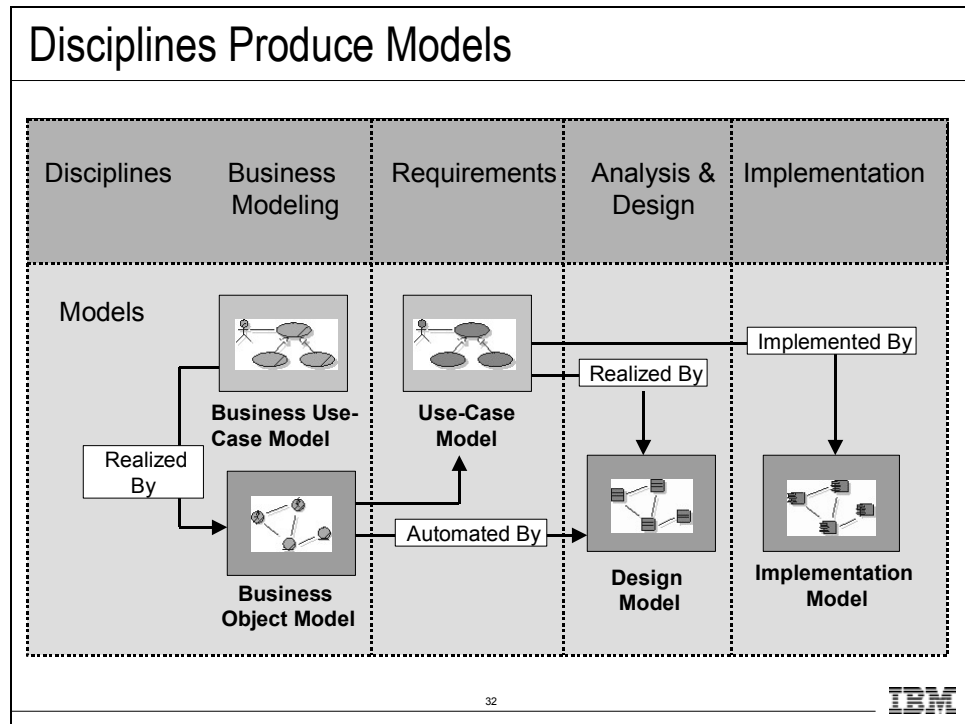


This graphic illustrates how phases and iterations (the time dimension) relate to the development activities (the discipline dimension). The relative size of the color area indicates how much of the activity is performed in each phase/iteration.

Each iteration involves activities from all disciplines. The relative amount of work related to the disciplines changes between iterations. For instance, during late Construction, the main work is related to Implementation and Test, and very little work on Requirements is done.

Note that requirements are not necessarily complete by the end of Elaboration. It is acceptable to delay the analysis and design of well-understood portions of the system until Construction because they are low in risk.

Disciplines Produce Models

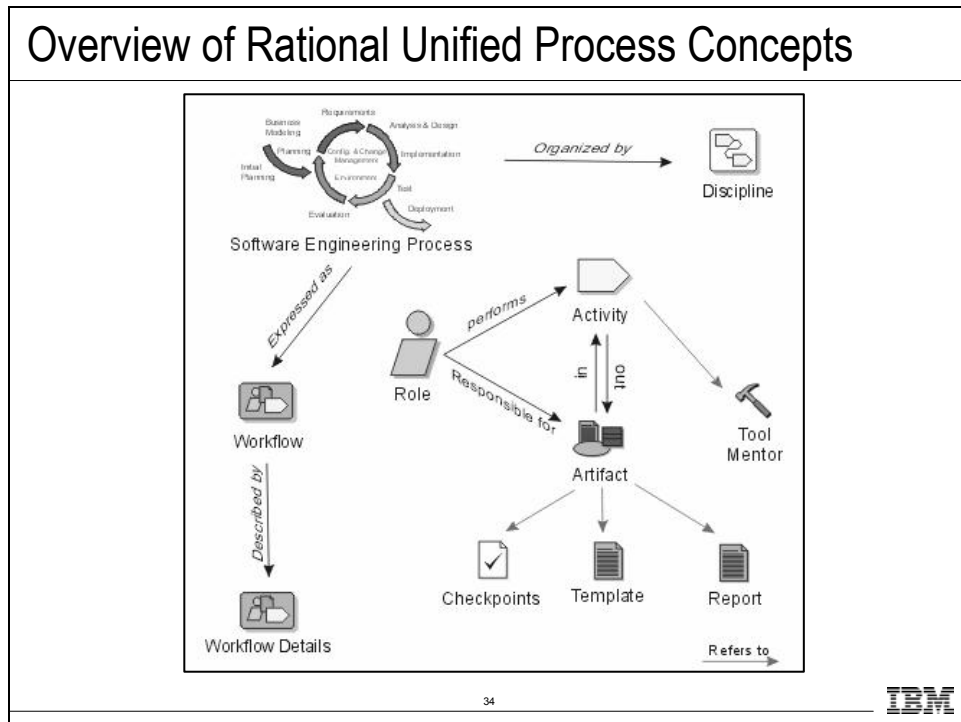


RUP is a model-driven approach. Several models are needed to fully describe the evolving system. Each major discipline produces one of those models. The models are developed incrementally across iterations.

- The **Business model** is a model of what the business processes are and of the business environment. It can be used to generate requirements on supporting information systems.
- The **Use-Case model** is a model of what the system is supposed to do and of the system's environment.
- The **Design model** is an object model describing the realization of use cases. It serves as an abstraction of the implementation model and its source code.
- The **Implementation model** is a collection of components and the implementation subsystems that contain them.

Test Suites are derived from many of these models.

Overview of Rational Unified Process Concepts



Basic Concepts in RUP

A **Role** defines the behavior and responsibilities of an individual or a set of individuals working together as a team.

An **Activity** is the smallest piece of work that is relevant.

- **Concepts** provide information which is important for understanding the workflow.
- **Work Guideline:** Work guidelines contain practical information about how to perform certain tasks.
- **Tool Mentor:** Most activities in RUP are supported by software-engineering tools.

Artifacts are the modeling constructs and documents that activities evolve, maintain, or use as input.

- **Checkpoints** provide a quick reference to help you assess the quality of the artifact.
- **Template:** There are a number of ready-to-use templates, which present documents and artifacts.
- **Artifact Guideline:** For most artifacts, RUP guidelines with detailed information about the artifact.
- **Report:** Models and model elements have reports associated with them. A report extracts information about models and model elements from a tool. A report presents an artifact or a set of artifacts.

Review

Review

- ♦ Best Practices guide software engineering by addressing root causes.
- ♦ Best Practices reinforce each other.
- ♦ Process guides a team on who does what, when, and how.
- ♦ Rational Unified Process is a means of achieving Best Practices.

